

1. Modelo relacional básico

◆ Qué es una tabla

Una tabla representa una **entidad**:

- **usuarios** → personas
 - **productos** → artículos
 - **pedidos** → relación entre ambos
-

◆ Clave primaria (PRIMARY KEY)

- Identifica de forma única cada fila
- Ejemplo:

id INT PRIMARY KEY

◆ Clave foránea (FOREIGN KEY)

Permite **relacionar tablas**

FOREIGN KEY (usuario_id) REFERENCES usuarios(id)

👉 Significa:

- Cada pedido pertenece a un usuario
 - No puedes insertar un pedido con un usuario inexistente
-

2. Relaciones entre tablas

◆ 1:N (UNO A MUCHOS)

Ejemplo:

- Un usuario → muchos pedidos

usuarios (1) ----- (N) pedidos

👉 Se implementa poniendo la FK en la tabla “muchos”:

pedidos.usuario_id

◆ N:1 (MUCHOS A UNO)

Ejemplo:

- Muchos pedidos → un producto

pedidos (N) ----- (1) productos

◆ N:M (MUCHOS A MUCHOS) (concepto)

👉 No lo usamos aquí directamente, pero importante:

- Se resuelve con una tabla intermedia
-

3. ¿Qué es un JOIN?

Un **JOIN** sirve para:

👉 Combinar datos de varias tablas relacionadas

Sin JOIN:

- Solo ves IDs

Con JOIN:

- Ves información real (nombre, producto, etc.)
-

4. INNER JOIN

◆ Concepto

Solo devuelve filas que **tienen relación en ambas tablas**

◆ Ejemplo real

```
SELECT u.nombre, p.nombre, pe.cantidad
FROM pedidos pe
JOIN usuarios u ON pe.usuario_id = u.id
JOIN productos p ON pe.producto_id = p.id;
```

👉 Resultado:

- Nombre del usuario
- Nombre del producto
- Cantidad

◆ Explicación mental

1. Empieza en **pedidos**
2. Busca el usuario correspondiente
3. Busca el producto correspondiente

5. LEFT JOIN

◆ Concepto

Devuelve **todo lo de la tabla izquierda**, aunque no tenga relación

◆ Ejemplo

```
SELECT p.nombre, pe.id
FROM productos p
LEFT JOIN pedidos pe ON p.id = pe.producto_id;
```

◆ Para qué sirve

👉 Detectar cosas que NO tienen relación

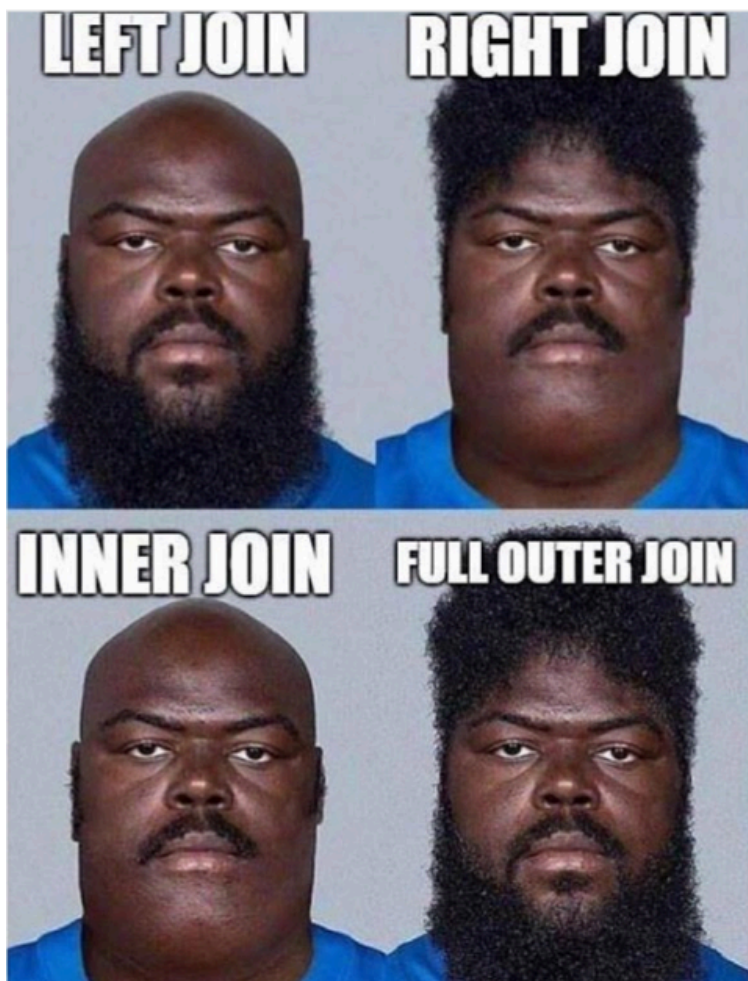
Ejemplo:

- Productos sin ventas

WHERE pe.id IS NULL

6. Diferencia clave

Tipo	Qué devuelve
INNER JOIN	Solo coincidencias
LEFT JOIN	Todo + coincidencias



7. GROUP BY y agregaciones

◆ SUM (muy importante)

```
SELECT producto_id, SUM(cantidad)
FROM pedidos
GROUP BY producto_id;
```

👉 Total vendido por producto

◆ COUNT

```
SELECT usuario_id, COUNT(*)
FROM pedidos
GROUP BY usuario_id;
```

👉 Número de pedidos por usuario

◆ Con JOIN (nivel real)

```
SELECT u.nombre, SUM(pe.cantidad)
FROM pedidos pe
JOIN usuarios u ON pe.usuario_id = u.id
GROUP BY u.nombre;
```

8. ORDER BY

```
ORDER BY total DESC;
```

👉 Para rankings:

- Más vendidos
 - Mejores clientes
-

9. WHERE con JOIN

```
SELECT *
```

```
FROM pedidos pe
JOIN usuarios u ON pe.usuario_id = u.id
WHERE u.activo = true;
```

👉 Filtrar después de relacionar

10. Subconsultas

◆ Ejemplo:

Producto más caro comprado por usuario

```
SELECT *
FROM pedidos
WHERE producto_id = (
  SELECT id
  FROM productos
  ORDER BY precio DESC
  LIMIT 1
);
```

Tablas con las que vamos a trabajar:

1. usuarios

```
CREATE TABLE usuarios (  
  id INT AUTO_INCREMENT PRIMARY KEY,  
  nombre VARCHAR(100) NOT NULL,  
  email VARCHAR(150) UNIQUE NOT NULL,  
  edad INT,  
  activo BOOLEAN DEFAULT TRUE  
);
```

2. productos

```
CREATE TABLE productos (  
  id INT AUTO_INCREMENT PRIMARY KEY,  
  nombre VARCHAR(100) NOT NULL,  
  precio DECIMAL(10,2) NOT NULL,  
  stock INT NOT NULL,  
  categoria VARCHAR(50),  
  activo BOOLEAN DEFAULT TRUE  
);
```

3. pedidos

```
CREATE TABLE pedidos (  
  id INT AUTO_INCREMENT PRIMARY KEY,  
  usuario_id INT,  
  producto_id INT,  
  cantidad INT NOT NULL,  
  fecha TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
  
  FOREIGN KEY (usuario_id) REFERENCES usuarios(id),  
  FOREIGN KEY (producto_id) REFERENCES productos(id)  
);
```

EJERCICIOS

1. Registrar un pedido con validaciones
 - Comprobar que el usuario está activo
 - Comprobar que el producto tiene stock
 - Insertar pedido
2. Listar pedidos de un usuario
 - Mostrar nombre producto + cantidad + fecha
 - Usar JOIN entre usuarios y productos
3. Reducir stock al hacer pedido
 - Cada vez que se inserta pedido, reducir stock
 - Usar dos queries separadas (sin transacción)
4. Top productos más vendidos
 - SUM(cantidad)
 - GROUP BY producto
5. Usuarios que más compran
 - Contar pedidos por usuario
6. Buscar productos por categoría y precio máximo
 - Filtros dinámicos con PreparedStatement
7. Desactivar usuarios inactivos
 - Usuarios sin pedidos → activo = false

8. Mostrar productos sin ventas
 - LEFT JOIN
 - Filtrar con NULL
9. Actualizar precio de productos por categoría
 - Subir o bajar precio en bloque
10. Historial completo de pedidos
 - Listado global con usuario, producto y cantidad
11. Validar edad mínima para comprar ciertos productos
 - Ejemplo: categoría "alcohol"
 - Lógica en Java, no en SQL
12. Eliminar pedidos antiguos
 - Borrar pedidos anteriores a una fecha dada
13. Ranking de usuarios por cantidad comprada
 - ORDER BY SUM(cantidad)
14. Mostrar el producto más caro comprado por cada usuario
 - Usar subconsulta o lógica en Java
15. Sistema de productos populares
 - Productos con más de X ventas
 - Marcar activo = true o false según popularidad